AI Assistant Log Classification

Pipeline Design & Proposed Approach

Nomura | AI Governance
May 2025 | Internal Document

Executive Summary

Nomura's AI Assistant generates 30,000–50,000 interaction logs daily. These logs contain high-value signals — what employees are doing, which departments are using AI for which tasks, and where governance gaps may exist. However, without a classification framework, this data remains unstructured and unactionable.

This document proposes a practical, cost-efficient pipeline to label, classify, and analyse these logs at scale — without requiring a pre-existing labeled dataset.

Goal: Classify every AI assistant log by Intent (what the user is trying to do) and Domain (which business area it relates to), enabling downstream analytics, governance reporting, and risk monitoring.

1. Problem Statement

At 30–50K logs per day, manual review is not feasible. The core challenge is a cold-start problem:

- A classification model requires labeled training data
- Labeled data requires either manual effort or an existing model
- Neither is currently available

Additionally, raw logs need preprocessing — a single user session may span multiple turns (uploading data, extracting insights, formatting output). These must be grouped intelligently before classification.

Scale: At 40K logs/day, even a 1% misclassification rate means 400 incorrect labels daily. Getting the pipeline architecture right is more important than optimising any single component.

2. Data & Preprocessing

2.1 Available Data Fields

Each log record contains the following fields:

Field	Description
session_id, row_id	Groups multi-turn conversations; row_id preserves turn order
prompt	User input — primary signal for intent classification
response	AI output — supporting signal for context
context	System or document context passed to the assistant
username, country	User identity and geography
emp_type	Employment type (e.g.public, private, vendor)
department	Business unit — strong prior for domain classification

2.2 Session-Level Grouping

Individual log rows do not tell the full story. A user interaction often spans multiple turns:

- Turn 1: User uploads a trade data file
- Turn 2: Asks for insight extraction
- Turn 3: Requests email formatting of the output

Classifying each row independently risks misidentifying intent. Instead, rows should be grouped by session_id and sorted by timestamp (row_id), then evaluated together to determine the primary intent of the session.

Rule: Short prompts (< 5 tokens) are merged with adjacent turns in the same session. Isolated short prompts with no context are filtered out as noise.

2.3 Metadata-Driven Signals

Structured fields like department and emp_type can dramatically improve classification confidence before any ML is applied:

- A log from HR department → strong prior for Domain: HR & People
- A log from Risk & Compliance → strong prior for Domain: Risk Management or Compliance
- emp_type = vendor → reduces likelihood of internal system/policy queries

Keyword and TF-IDF analysis on department-segmented logs can surface domain-specific vocabulary, enriching the taxonomy definitions in the next step. This metadata enrichment reduces the classification burden on the model significantly.

3. Taxonomy Definition

Classification requires a well-defined taxonomy before any labeling begins. Two orthogonal dimensions are proposed:

3.1 Intent Taxonomy (What is the user doing?)

Code	Intent	Description
I01	Summarisation	Condensing a document, report, or conversation into key points
I02	Formatting	Restructuring or reformatting content without changing meaning
I03	Drafting	Writing emails, memos, reports, or documents from scratch
I04	Explanation	Explaining a concept, term, regulation, or process
I05	Data Extraction	Pulling specific fields, numbers, or facts from a document
I06	Analysis	Interpreting data, identifying trends, comparing scenarios
I07	Code Generation	Writing, debugging, or refactoring code or queries
I08	Translation	Language translation or simplifying technical jargon
I09	Q&A / Lookup	Direct factual question expecting a specific answer
I10	Review & Feedback	Proofreading, critiquing, or improving existing content

3.2 Domain Taxonomy (Which business area does it relate to?)

Code	Domain	Description
D01	Markets & Trading	Market data, trade execution, pricing, instruments
D02	Risk Management	Credit/market risk, VaR, stress testing, exposure limits
D03	Compliance & Regulatory	KYC, AML, MiFID II, SEBI, SEC reporting obligations
D04	Finance & Accounting	P&L, reconciliation, financial statements, cost allocation
D05	HR & People	Leave, payroll, performance reviews, hiring policies
D06	Technology & IT	System access, infrastructure, code, debugging, tools
D07	Research & Analytics	Equity reports, macro analysis, data science workflows
D08	Legal & Contracts	NDAs, agreements, legal opinions, entity structures
D09	Operations	Settlement, custody, middle office, back office processes
D10	General / Admin	Meetings, emails, scheduling, unclassified office tasks

Note: Each log receives one primary Intent and one primary Domain. Where ambiguity exists, department metadata is used as a tiebreaker. D10 General/Admin is a last resort — over-use of this category indicates a gap in taxonomy coverage.

3.3 Taxonomy Validation via Clustering

Before finalising the taxonomy, exploratory clustering will be run on 15–20 days of log data. This serves two purposes:

- Validate that proposed categories reflect actual usage patterns in Nomura's context
- Surface any missing categories that real usage reveals but the initial taxonomy does not cover

HDBSCAN is preferred over K-Means as it does not require specifying the number of clusters in advance and handles noise well. Clustering output will be reviewed manually and mapped to taxonomy codes — it is an input to taxonomy design, not a classification method in production.

4. Classification Pipeline

4.1 End-to-End Flow

Step 1 Taxonomy Definition Define Intent × Domain categories with definitions, keywords, and prototype examples

↓

Step 2 Data Preprocessing Group by session, sort turns, merge short prompts, enrich with department/emp_type signals

↓

Step 3 Hybrid Semantic Scoring Score each log against taxonomy using prototype embeddings (50%), keyword match (30%), definition similarity (20%)

↓

Step 4 Threshold Decision High confidence → auto-label. Low confidence → route to LLM for classification

↓

Step 5 LLM Fallback SLM/LLM classifies uncertain logs with reason field — enables quality review and false positive removal

↓

Step 6 Manual Spot-Check Sample verification on auto-labeled and LLM-labeled data to validate quality before training

↓

Step 7 Classifier Training Train lightweight Intent + Domain classifiers on labeled dataset. Replaces hybrid pipeline in production

5. Hybrid Semantic Scoring

5.1 Why Not Pure Keyword or Definition Matching?

Simple keyword matching fails on paraphrasing — 'restructure this document' would not match a keyword list built around 'format', 'reformat', 'beautify'. Pure definition similarity is too broad — 'transformation of structure without changing meaning' overlaps with summarisation and editing in embedding space.

5.2 Prototype-Based Embedding

For each taxonomy category, 5–10 real example prompts (prototypes) are manually written. These are embedded and stored as a category anchor:

- Prototype examples for I02 Formatting: "Convert this email into bullet points", "Reformat this JSON", "Turn this report into a markdown table"
- Incoming log is embedded and compared against all prototype sets
- Similarity score = max similarity to any prototype (not average — to preserve strong signal)

Why max not average: Averaging prototype embeddings dilutes the cluster. Taking the max similarity finds the closest match and avoids penalising categories where only one prototype is highly relevant.

5.3 Scoring Formula

Component	Weight	Rationale
Prototype Similarity	50%	Highest signal — real examples anchor the embedding space precisely
Semantic Keyword Match	30%	Keywords embedded (not exact match) to handle paraphrasing
Definition Similarity	20%	Broad context signal; lower weight to avoid noise

Keywords are also embedded (not matched exactly), so semantically similar words ('restructure' ≈ 'reformat') contribute to the keyword score without requiring exhaustive keyword lists.

5.4 Threshold Calibration

The boundary between high-confidence auto-labeling and LLM fallback will be calibrated using a pilot batch of 200 logs labeled by LLM (ground truth). Grid search over threshold values identifies the setting that maximises precision while keeping LLM call volume within budget. The same pilot batch is used to tune the 50/30/20 weights.

6. LLM Fallback Strategy

6.1 Why LLM for Low-Confidence Cases?

LLMs understand intent contextually in a way that embedding similarity cannot. For ambiguous logs — where a query could plausibly belong to two categories — LLM classification with a reason field produces high-quality labels that are also human-reviewable.

6.2 Cost Management

The key risk with LLM-based labeling is unpredictable cost at scale. Three mitigations are applied:

- Semantic scoring filters out high-confidence cases — LLM only sees genuinely ambiguous logs (estimated 20–30% of volume based on pilot)
- Prompt truncation: only the first 300 characters of the prompt field are sent, which captures intent in the vast majority of cases and reduces token cost by 60–70%
- Hard budget cap: a maximum token/cost limit is enforced per batch run, with alerting if rare categories remain under-represented

6.3 Rare Category Handling

Random sampling will under-represent rare domains (e.g. HR, Legal). A keyword pre-filter is applied before LLM calls to surface candidate logs for rare categories specifically:

- HR candidates: 'leave', 'salary', 'appraisal', 'payroll', 'performance review'
- Legal candidates: 'NDA', 'agreement', 'clause', 'indemnity', 'entity'
- These candidates are pooled with random samples to ensure ≥ 20 examples per category in the labeled dataset

7. Labeling Methods: Comparison

Method	Pros	Cons
LLM / SLM Only	High accuracy, includes reason field for review	Cost unpredictable; need to know how many calls required per category upfront
Semantic Only	Zero cost, fully scalable, easy to iterate	Lower accuracy on ambiguous prompts; requires well-tuned definitions and prototypes
Hybrid (Proposed)	Cost-efficient; high accuracy where it matters; scalable	Requires threshold calibration using pilot data

Decision: Hybrid approach is recommended. Semantic scoring handles the majority of logs at zero cost; LLM is reserved for genuinely ambiguous cases. This yields high-quality labels at predictable cost.